Technical Note #2
On the Temporal Structure of Distributed Pipelines

Charles Colella
contact@charlescolella.com

May 2026

Abstract

In systems that consume streams to make time-sensitive decisions, timing variations in the pipeline can change what the downstream system observes. Common delay summaries such as mean, median, or P95 describe delay levels, but not how delays are organized over time. This note treats the system as a transformation from input time to output time and studies timing gaps, local stretching and compression, and persistent temporal episodes as parts of the same temporal structure. We apply the framework to a real-time market-data pipeline, characterize several temporal transformations, and show how they are linked to the interpretation of a downstream multi-exchange signal.

1 Introduction

Two online pipelines can ingest the same events and still produce different signals solely because they make those events visible at different times. This matters in systems that consume streams to make time-sensitive decisions. For instance, a trading system reacts to price changes, a monitoring system to alerts, and a robotic system to sensor updates. In all these cases, the value of an event depends both on its content and on when it becomes available.

This makes the timing introduced by the system an object to characterize. In a distributed pipeline, timestamps may come from different clocks, and their comparison is affected by synchronization and observation errors. Once these timestamps are available, the question is how to read the resulting timing behavior. Marginal delay summaries give useful information about delay levels, but they lose the order of the observations. They can therefore miss structured temporal effects such as accumulation, stretching, compression, or persistent stale periods.

The central idea of this note is to view the pipeline as a temporal transformation from an input stream to an output stream. Delay level, staleness, and local stretching or compression then appear as different aspects of how the system reshapes time between input and output. This note is the second in a broader series[1]. Technical Note I [1] analyzed the design of a real-time market-data pipeline processing streams from multiple exchanges during the U.S. market open. Here, we use the same setting to characterize the temporal transformation induced by the pipeline and to study its effect on downstream signals.

We first define a framework for describing temporal transformations in distributed systems. We then apply it to the 270 million events collected in Technical Note I to characterize how timing is reshaped across streams and regimes. This analysis provides several views of the transformation, including cross-stream dispersion, local stretching and compression, and the persistence of same-sign distortion episodes.

The main empirical test is a multi-exchange dislocation signal. This signal compares prices observed on two exchanges at matched times, so it depends directly on the temporal reconstruction of the streams. We use it to ask whether the temporal structure induced by the pipeline remains an internal systems property, or whether it changes the signal observed by downstream logic. We then empirically link compression episodes, persistent runs, and inter-stream timing divergence to downstream signal changes. This motivates a final discussion of the practical consequences for real-time systems.

2 Analytical framework

This section defines the framework used to describe temporal transformations in a distributed system. The goal is to characterize, in a unified way, timing behaviors such as global shifts, local stretching or compression, and persistent temporal episodes. We view the pipeline as a map from input event times to output event times. In a distributed system, this map is observed through imperfect timestamps and losses.

2.1 Timestamp basis

We call a timestamp basis the time reference used to record a timestamp at a given observation point. To separate timestamp values from measurement error, we assume that each timestamp basis a is related to a latent reference time. For instance, a recorded timestamp on basis a is written as

$$a_i = t_i^a + e_a(i), \tag{1}$$

where t_i^a is the corresponding event time expressed in the latent reference time, and $e_a(i)$ is the timestamp

[1] The technical series is accompanied by the application code, infrastructure configuration, and deployment materials used to run the live pipeline [2].

error of basis a relative to that reference. We assume that timestamp errors are bounded on each basis. For any timestamp recorded on basis a,

$$|e_a(i)| \le \eta_a. \tag{2}$$

where η_a denotes an elementary error bound for timestamp basis a.

2.2 System definition

We begin with the simplest case, namely a loss-free system that induces a transformation between a single input timestamp basis and a single output timestamp basis. Extensions to losses and multiple timestamp bases are introduced afterwards.

We consider a system S that takes as input an ordered sequence of observations timestamped on basis r, over a finite observation window. The sequence is represented by $(r_i)_{0\le i\le N}$, where r_i is the timestamp of the i-th observation. Let o denote the timestamp basis on which the corresponding output observations are recorded. Since the system is assumed to be loss-free, we can match each input observation with exactly one output observation. This allows us to define the system transformation

$$T^{(S)}_{r\to o} : (r_i)_{0\le i\le N} \mapsto (o_i)_{0\le i\le N},$$

where o_i is the timestamp, on basis o, of the output observation matched with the i-th input observation. This indexing is defined by the matching itself, so output reordering does not affect the index assigned to each matched observation.

2.3 Finite-difference metrics

We now introduce a family of finite-difference metrics for reading the transformation $T^{(S)}_{r\to o}$. They provide structured views of how the system modifies the timing of the stream. The order-0 metric measures the pointwise timing gap between matched input and output observations. First-order metrics measure how this gap changes over an event horizon k, and therefore expose local stretching or compression of the stream. Higher-order metrics measure how these local distortions themselves vary across the stream. In this sense, the metrics below separate different aspects of the same timestamp transformation: delay level, spacing distortion, and changes in distortion. Because the recorded timestamps are subject to the error model introduced above, each metric is also accompanied by a conservative error bound.

Order-0: timing gap. We define the timing gap of the i-th matched observation as

$$D^{(0)}_i = o_i - r_i. \tag{3}$$

Applying the timestamp model of Section 2.1, we obtain

$$D^{(0)}_i = (t^o_i - t^r_i) + (e_o(i) - e_r(i)). \tag{4}$$

The first term is the latent-reference timing gap, and the second term is the observation error. From Equation (2), the observation error is bounded by $\eta_r + \eta_o$.

The sequence $D^{(0)}$ is the basic observable of the temporal transformation. It measures the pointwise separation between matched input and output timestamps. Under the loss-free matching assumptions, this sequence fully describes the observed input-output timing map. The derived metrics introduced below then read this sequence at different scales, separating its level, local variation, and longer-horizon dynamics.

Order-1: step-k timing-gap distortion. For any event step $1 \le k \le N$, we define the order-1 metric by

$$D^{(1,k)}_i = D^{(0)}_{i+k} - D^{(0)}_i, \qquad i \le N - k. \tag{5}$$

This finite difference measures how the timing gap changes over the next k matched transitions. Positive values correspond to local expansion of the output stream relative to the input stream, and negative values correspond to local contraction. Applying the timestamp model of Section 2.1, we obtain

$$\begin{aligned} D^{(1,k)}_i = {} & \big[(t^o_{i+k} - t^o_i) - (t^r_{i+k} - t^r_i)\big] \\ & + \big[(e_o(i+k) - e_o(i)) \\ & - (e_r(i+k) - e_r(i))\big]. \end{aligned} \tag{6}$$

The first term is the latent spacing distortion over horizon k, and the second is an observation-error term bounded by $2\eta_r + 2\eta_o$. This is the metric used below to identify local stretching and compression episodes in the transformed stream. Similar finite-difference timing quantities are commonly used in packet-network measurements to characterize timing instability along network paths [3].

Order-2: second-order timing-gap distortion. For any event step $1 \le k \le \lfloor N/2 \rfloor$, we define the order-2 metric as

$$D^{(2,k)}_i = D^{(1,k)}_{i+k} - D^{(1,k)}_i, \qquad i \le N - 2k. \tag{7}$$

This finite difference measures how the spacing distortion changes between two successive horizons of length k. It is therefore a second-order view of the temporal transformation, useful for detecting changes or alternations in local stretching and compression. Applying the timestamp model of Section 2.1, we obtain

$$\begin{aligned} D^{(2,k)}_i = {} & \big[((t^o_{i+2k} - t^o_{i+k}) - (t^o_{i+k} - t^o_i)) \\ & - ((t^r_{i+2k} - t^r_{i+k}) - (t^r_{i+k} - t^r_i))\big] \\ & + \big[(e_o(i+2k) - 2e_o(i+k) + e_o(i)) \\ & - (e_r(i+2k) - 2e_r(i+k) + e_r(i))\big]. \end{aligned} \tag{8}$$

The first term is the latent second-order spacing distortion, and the second is an observation-error term bounded by $4\eta_r + 4\eta_o$. Higher-order metrics can be defined by successive differencing. In practice, their error bounds grow with the order and their interpretation becomes less direct, so the empirical analysis below focuses mainly on Order-0 and Order-1 quantities.

A useful property of these metrics is additivity across consistently matched observation points. If the transformation from timestamp basis a to timestamp basis

c is observed through an intermediate basis b, and if the sequences on a, b, and c share the same index-wise matching over the same event horizon k, then

$$D_i^{(n,k)}(a \to c) = D_i^{(n,k)}(a \to b) + D_i^{(n,k)}(b \to c). \quad (9)$$

This makes it possible to attribute temporal transformations across successive parts of a processing chain when additional observation points are available.

2.4 Refined error bounds

The bounds above are conservative because they only use absolute timestamp-error bounds. In real systems, timestamp errors often evolve more regularly. One possible refinement is to assume that, on each timestamp basis c, the timestamp error has bounded variation with respect to the latent reference time:

$$|e_c(j) - e_c(i)| \le \lambda_c |t_j^c - t_i^c|, \quad (10)$$

then the error bound of first-order metrics depends on the variation of the clock error over the event horizon, rather than on its absolute magnitude. Under this assumption, the order-1 observation error is bounded by

$$\lambda_o |t_{i+k}^o - t_i^o| + \lambda_r |t_{i+k}^r - t_i^r|, \quad (11)$$

which may be much sharper than $2\eta_o + 2\eta_r$ when clock errors vary slowly. Higher-order bounds can be refined in the same way by applying the corresponding finite-difference operator to the error terms.

2.5 Extension to losses

The framework above assumes that the input and output sequences admit a loss-free index-wise matching over the observation window. In practical systems, some events may be missing because of drops, coalescence, or incomplete capture. We represent this with an observation indicator $R_i \in \{0, 1\}$, where $R_i = 1$ means that the event is retained and can be matched. The timing gap is then defined on the retained index set

$$\mathcal{I}_{\text{obs}} = \left\{ i \;\middle|\; \begin{array}{l} 0 \le i \le N, \\ R_i = 1 \end{array} \right\}. \quad (12)$$

Higher-order metrics require all indices involved in the finite difference to be retained. For an order-j metric with event step k, the admissible set is therefore

$$\mathcal{I}_{\text{obs}}^{(j,k)} = \left\{ i \;\middle|\; \begin{array}{l} 0 \le i \le N - jk, \\ R_{i+mk} = 1, \ \forall m \in \{0, \dots, j\} \end{array} \right\}. \quad (13)$$

Under losses, the metrics are conditional observables on the retained matched subsequence. This matters for interpretation. If losses are independent of the temporal quantities being studied, the retained subsequence may remain representative of the latent stream. If losses concentrate during stressed regimes, large timing transformations may be underrepresented.

Correcting this bias requires additional assumptions on the observation mechanism. In particular, one must relate the unobserved values of $D_i^{(j,k)}$ outside $\mathcal{I}_{\text{obs}}^{(j,k)}$ to the values observed inside it. This relation cannot be inferred from the retained subsequence alone. Unless an explicit loss model is available, finite-difference metrics under losses should therefore be interpreted on their admissible support and reported with a description of that support, such as the retained fraction over a finite window.

2.6 Extension to multiple bases

The definitions above use one input timestamp basis and one output timestamp basis. In a distributed system, several observation points may coexist on both sides of the pipeline. Each observation point can provide its own timestamp basis and therefore its own view of the input-output transformation.

Without additional assumptions on the relation between timestamp errors across bases, each input-output pair must be treated as a separate transformation path. For an input basis r_p and an output basis o_q, we denote this path by $T_{r_p \to o_q}^{(S)}$.

The finite-difference metrics can then be computed separately on each path. Their error bounds are determined by the two bases used in that path. Comparisons across paths remain possible, but they should be made through derived observables defined path by path, such as differences between timing-gap metrics or higher-order metrics.

3 Case study

We now apply this framework to a case study based on the distributed market data pipeline already introduced in Technical Note I [1]. In this section, we first use a small set of observables based on the Order-0 and Order-1 metrics to show the different questions they address. We then use an example based on multi-stream matching to illustrate how these effects may propagate to downstream derived signals.

3.1 Pipeline and observation setup

Our case study is a real-time market-data pipeline used to transform exchange updates into derived signals. The system receives updates for multiple symbols from multiple exchanges. For each exchange-symbol pair, it reconstructs a view of the buy and sell orders currently available in the market, together with their quantities (a Level 2 order book). It then computes derived features from the successive states of this order book. In this study, a stream is defined as an exchange-symbol-feature triplet. For each stream, we observe an input timestamp sequence and an output timestamp sequence. Our object of interest is the temporal relation between these two sequences, that is, how the pipeline transforms temporal structure from input to output.

3.2 Data sources and conditions

The experiments use public spot cryptocurrency market data from Binance and KuCoin. The inputs are incremental order book updates. Binance updates are

quantized at 50 ms, and KuCoin updates are not visibly quantized. The pipeline ingests the 467 symbols listed on both exchanges, that is, up to 934 live order books in total. The outputs are six derived features computed from the order book state, covering price, spread, and liquidity-related quantities. Their formal definitions are provided in Technical Note I [1].

The system is first run on live data from both exchanges over a one-hour measurement window spanning March 3, 2026 from 14:00 to 15:00 UTC. The processing infrastructure runs on a fixed 8-node cluster in the Tokyo region. All input updates from both exchanges enter the pipeline through the same reception node and therefore share a common input clock. Output records are dynamically distributed across 5 nodes, so no single common output clock can be assumed. After the experiment, snapshots of the infrastructure, running services, and stored outputs are collected, and the analysis is carried out offline.

Over this one-hour run, Technical Note I [1] identified two distinct regimes on both exchanges. Before 14:30, load remains comparatively moderate and stable. After 14:30, it increases markedly and exhibits more frequent spikes. This transition coincides with the start of the main U.S. cash-equity trading session. Table 1 reports the number of feature records observed at the pipeline output for each exchange–regime pair.

Exchange	Calm regime	High regime	Total
KuCoin	70.7M	165.6M	236.4M
Binance	15.7M	25.1M	40.8M

Table 1: Feature counts at pipeline output.

As shown in Table 1, this one-hour window contains enough data for a detailed analysis of temporal behavior. The two distinct regimes also make it possible to compare the metrics under different load conditions.

3.3 Framework instantiation

We now instantiate the framework on the experimental setting. This requires specifying the timestamp bases, the practical timestamp-error bounds, and the loss mechanisms used in the empirical analysis.

Timestamp bases. For each stream s, defined as an exchange–symbol–feature triplet, the input timestamp is the update reception time, denoted $r^{(s)}$. The output timestamp is the feature emission time, denoted $o^{(s)}$, recorded after feature computation and before the record leaves the system.

In this run, all input updates are timestamped at the same reception point, so we write $r^{(s)} = r$ for all streams. Output records, however, may be emitted by different nodes, and a stream is not assumed to remain attached to a fixed physical output clock over the whole observation window. We therefore keep the stream-dependent notation $o^{(s)}$. This yields 2802 output timestamp sequences per exchange, each treated as a distinct analytical output path.

Practical error bounds. Each node synchronizes its clock to a common NTP reference. Appendix A.1 estimates the practical absolute timestamp error relative to this reference as $\hat{\eta} = 784\,\mu$s. For Order-0 timing-gap measurements, this gives the bound $2\hat{\eta} = 1568\,\mu$s.

For Order-1 metrics, this absolute bound is overly conservative because the metric depends on changes in timestamp error over a short event horizon. We therefore use the refined bound from Section 2.4. The assumption is that node-clock errors have a bounded rate of change over the observation window. Appendix A.1 estimates the corresponding coefficient as $\hat{\lambda} = 3.86\,\mu$s/s. This estimate relies on the clock-synchronization measurements and assumes clock discipline, no clock steps, and negligible timestamping overhead.

Losses on nominal segments. Our analysis focuses on the pipeline in its nominal regime, that is, during online event processing. This regime may be interrupted by recovery phases, such as snapshot-related recovery events described in Technical Note I [1]. For each stream, the metrics are therefore computed separately on each nominal segment, and the resulting values are then aggregated into final distributions.

Within these nominal segments, some input observations may still have no matched output observation. In this study, two loss sources must be separated. Some losses come from the pipeline itself, for example when updates are dropped under high load before feature emission. Others come from the offline reconstruction of runs from large stored datasets, which may introduce additional unmatched observations.

Exchange	Loss type	Count	Ratio (%)
KuCoin	Offline reconstruction	158K	0.067
KuCoin	Pipeline	99K	0.042
KuCoin	No loss	236M	99.891
Binance	Offline reconstruction	143K	0.350
Binance	Pipeline	30K	0.074
Binance	No loss	40M	99.573

Table 2: Loss counts aggregated over nominal segments by exchange.

Offline reconstruction losses arise during offline metric computation. Output features are first stored in a distributed database and then analyzed offline. Finite-difference metrics require access to the successor relation within each stream. In practice, these relations are computed independently inside one-minute buckets. As a result, transitions crossing a bucket boundary are not represented.

As discussed in Section 2.5, the reported metrics remain conditional on the observed support. Table 2 nevertheless shows that the fraction of unmatched transitions remains limited. The two loss mechanisms do not have the same interpretation: pipeline losses are part of the live system behavior and may be associated with periods of higher load, in which case they may underrepresent the strongest temporal transformations. Offline reconstruction losses, by contrast, are

exogenous to the pipeline behavior itself and follow a regular pattern by construction, so their effect is easier to account for.

3.4 Order-0 derived metrics

We begin the empirical reading of the temporal transformation. The analysis focuses on KuCoin, whose higher update rate makes backlog and related temporal effects more visible, as shown in Technical Note I [1]. Since the case study contains a large number of streams, we analyze the metrics through offline distributional summaries rather than individual time series.

The first question is whether the pipeline changes the delay level observed by different streams, and whether this effect changes between the calm and high regimes. To answer this, we summarize $D^{(0)}$ using the mean, median, and P95.

Figure 3 shows the cross-stream distributions of these summaries in the calm and high regimes. In both regimes, the median remains concentrated around a few milliseconds, so the typical delay stays contained for most streams. The mean is higher than the median, and the P95 is larger and more dispersed. Thus, even before comparing regimes, the timing-gap distribution is visibly right-skewed, with tail behavior varying substantially across streams.

The second question is whether the high regime changes all streams in a similar way, or instead increases the heterogeneity between streams. Table 3 answers this by comparing cross-stream relative dispersion between regimes. The high regime increases relative dispersion for all three summaries. The effect is strongest in the central and upper parts of the cross-stream distribution, which means that the main Order-0 change is not simply a uniform increase in delay. It is also a widening of stream-specific tail latency.

Metric	$\frac{\mathrm{IQR}_{rel}^{\mathrm{high}}(X)}{\mathrm{IQR}_{rel}^{\mathrm{calm}}(X)}$	$\frac{U_{rel}^{\mathrm{high}}(X)}{U_{rel}^{\mathrm{calm}}(X)}$	$\frac{L_{rel}^{\mathrm{high}}(X)}{L_{rel}^{\mathrm{calm}}(X)}$
Mean $D^{(0)}$	3.018755	2.254627	1.128414
Median $D^{(0)}$	2.381858	3.092124	1.538217
P95 $D^{(0)}$	2.119827	1.570365	1.016680

Table 3: Ratios of cross-stream relative dispersion between regimes. IQR_{rel}, U_{rel}, and L_{rel} are normalized by Q_{50} and measure central, upper-tail, and lower-tail spread respectively.

This view is useful but still marginal. It summarizes the distribution of pointwise timing gaps and does not describe how delay variations are arranged over the stream. The next step is therefore to analyze first-order structure, where compression and stretching runs expose whether timing-gap changes are isolated or organized into persistent local regimes.

3.5 Order-1 derived metrics

We first ask whether local timing-gap changes are biased toward stretching or compression. We measure this using the stretching-transition share, defined as the fraction of observed transitions for which $D_i^{(1,1)} > 0$. Near-zero values may have an uncertain sign, but the refined first-order uncertainty bound is small relative to the aggregate effects reported below. A value close to 0.5 means that stretching and compression occur with comparable frequency.

Figure 4 shows that the median share is close to 0.5 in both regimes. The distribution is slightly more extended on the compression side, which indicates that some streams contain more compression transitions, but there is no clear regime-level shift in this first-order balance. This transition share answers only a marginal question: how often each direction occurs. It does not tell us whether these changes are isolated or temporally organized. We therefore ask a second question: when stretching or compression occurs, does it persist over several consecutive transitions?

To answer this, we group consecutive transitions with the same sign into episodes. A stretching episode is a maximal same-sign run of positive $D_i^{(1,1)}$, and a compression episode is a maximal same-sign run of negative $D_i^{(1,1)}$. Formally, an episode $E \in \mathcal{E}^+$ (resp. $E \in \mathcal{E}^-$) is a maximal contiguous sequence of indices $E = \{m, \dots, m+j-1\}$ such that $D_i^{(1,1)} > 0$ (resp. $D_i^{(1,1)} < 0$) for all $i \in E$. Its length, measured in transitions, is $L(E) = j$. We call an episode persistent when it contains at least three consecutive transitions. Let $\mathcal{E}^{\pm}_{\geq 3} = \{E \in \mathcal{E}^{\pm} : L(E) \geq 3\}$. For each sign, we compute the persistent-run share

$$P^{\pm}_{\geq 3} = \frac{\sum_{E \in \mathcal{E}^{\pm}_{\geq 3}} L(E)}{\sum_{E \in \mathcal{E}^{\pm}} L(E)}. \tag{14}$$

This quantity answers the question: among all stretching or compression transitions, what fraction belongs to a persistent same-sign episode? Figure 4 shows that compression transitions are more often part of persistent episodes than stretching transitions. This asymmetry is visible in both regimes. Thus, even though the overall number of stretching and compression transitions is broadly balanced, compression appears more clustered in time.

The next question is how long these persistent episodes usually last. For each stream, we summarize the distribution of persistent episode lengths using $p_{50}(L(E) \mid E \in \mathcal{E}^{\pm}_{\geq 3})$ and $p_{95}(L(E) \mid E \in \mathcal{E}^{\pm}_{\geq 3})$. The median describes the typical persistent episode, while the p_{95} describes the upper tail of episode duration within the stream. Table 4 shows that persistent episodes remain short in both directions. Compression episodes are slightly longer than stretching episodes, especially in the upper tail, but the effect remains limited in absolute length.

Finally, we ask whether persistent episodes are also the strongest ones. This is a distinct question: an episode may be persistent because the stream is regularly transformed in the same direction, without necessarily having large per-transition magnitude. We therefore compare the typical magnitude and within-stream

Dir.	$Q_{.05}(p_{50})$	$Q_{.95}(p_{50})$	$Q_{.05}(p_{95})$	$Q_{.95}(p_{95})$
−	3.0	4.0	5.0	8.0
+	3.0	3.0	4.0	6.0

Table 4: Cross-stream quantiles of within-stream summaries $p_{50}(L(E) \mid E \in \mathcal{E}^{\pm}_{\geq 3})$ and $p_{95}(L(E) \mid E \in \mathcal{E}^{\pm}_{\geq 3})$.

variability of persistent and non-persistent episodes.

Figure 5 shows that non-persistent episodes have higher typical magnitudes than persistent ones in both regimes. Their standard deviation is also larger, which indicates stronger within-stream variability. Thus, the largest first-order distortions are mostly short-lived.

Persistent episodes show the opposite behavior. They have lower typical magnitudes and lower within-stream variability, which is consistent with more regular same-sign transformations. However, their median magnitudes remain more heterogeneous across streams. Persistent runs therefore do not correspond to the strongest local distortions; rather, they identify streams where smaller distortions are organized more regularly over consecutive transitions.

The Order-0 and Order-1 summaries therefore provide complementary readings of the temporal transformation. Table 5 summarizes the practical question addressed by each summary and the corresponding empirical reading in this case study.

3.6 Multi-exchange illustration

3.6.1 Setup

The preceding sections characterized timing transformations within individual streams. We now use a multi-exchange signal to illustrate how such temporal transformations can affect a downstream quantity constructed from several streams.

A common question in market-data processing is how the price of the same symbol on Binance stands relative to its price on KuCoin at the same moment. Since the two exchanges do not update at exactly the same instants, and Binance updates follow a fixed 50 ms cadence in the observed stream, we use a simple alignment rule: each Binance price update m_j^{Bin} is matched with the most recent KuCoin price available at that time. We define the dislocation as

$$d_j = 10^4 \left(\log m_j^{\mathrm{Bin}} - \log m_{i(j)}^{\mathrm{Kuc}} \right),$$

where $i(j)$ denotes the index of the latest KuCoin update available at the time of the j-th Binance update. The signal observed downstream may itself be affected by the pipeline, since the latter changes the timing at which updates become available for alignment. To evaluate this bias, we compare the dislocation computed at pipeline input time with the dislocation computed at pipeline output time.

Formally, let B_i be a Binance event, with input timestamp r_i^B and output timestamp o_i^B, and let K_j and K_{j+1} be two consecutive KuCoin events such that $r_j^K < r_i^B \leq r_{j+1}^K$. At output time, we use the latest-available matching rule: the Binance output event o_i^B is associated with the most recent KuCoin output event available at that time. A mismatch occurs when this output-time rule does not select the output corresponding to K_j. Under these definitions, a necessary and sufficient condition for the output-time rule to preserve the input-time match is $o_j^K < o_i^B \leq o_{j+1}^K$, or equivalently $0 < o_i^B - o_j^K \leq o_{j+1}^K - o_j^K$. Using the definition of the Order-0 timing gap, we obtain

$$0 < \frac{\left(r_i^B - r_j^K\right) + \left(D_i^B - D_j^K\right)}{\left(r_{j+1}^K - r_j^K\right) + \left(D_{j+1}^K - D_j^K\right)} \leq 1, \qquad (15)$$

where D_i^B and D_j^K denote the Order-0 timing gaps on the Binance and KuCoin streams. This ratio measures the relative position of the Binance event within the KuCoin interval $[K_j, K_{j+1}]$ after transformation by the pipeline, and a mismatch occurs exactly when this position leaves the interval $]0, 1]$. The numerator is the transformed offset of the Binance event from the left KuCoin boundary and is driven by the relative Order-0 timing gaps on the two streams. The denominator is the transformed KuCoin interval length and is driven by the Order-1 distortion on the KuCoin stream.

This expression shows how the error observed on the downstream signal is linked to the temporal transformations induced on each stream and to their interaction. It also shows that these effects matter in practice when inter-arrival gaps are comparable to, or smaller than, the magnitude of those transformations.

3.6.2 Empirical results

We compute the dislocation signal on the data collected by the pipeline over the observation window. The analysis covers the 467 symbols observed on both Binance and KuCoin. For each symbol, the same-symbol Binance–KuCoin matching is computed twice, using reception timestamps on basis r and output timestamps on basis $o^{(symbol,exchange)}$.

To avoid stale comparisons, we discard matches where the selected KuCoin update is too old relative to the usual KuCoin update scale for that symbol. For both the input-time and output-time matchings, a KuCoin price is considered stale if its age at the Binance update time exceeds $10\,\mathrm{med}_j(\Delta_j^K) + \varepsilon$, where Δ_j^K denotes the KuCoin inter-arrival gap used for that matching and ε is an observation-error tolerance bounded by $2\hat{\eta} = 1568\,\mu$s. If the KuCoin price is stale in either matching, the Binance update is excluded.

We first ask whether the mismatch rate is reflected in simple stream-level summaries of the temporal transformation. Equation (15) shows that mismatches become more likely when the KuCoin interval is small relative to the temporal transformation introduced by the pipeline. For each symbol, we summarize this scale relation by

$$\rho_K = \frac{\mathrm{med}_j(D_{K,j}^{(0)})}{\mathrm{med}_j(r_{j+1}^K - r_j^K)},$$

Metric	Summary	Question	Finding
$D^{(0)}$	Median, mean, P95	How large is the pointwise timing gap across streams and regimes?	Typical delay remains contained, but the distribution is right-skewed and tail behavior varies across streams.
$D^{(0)}$	Relative dispersion ratios	Does the high regime shift all streams similarly, or increase cross-stream heterogeneity?	The high regime mainly widens cross-stream heterogeneity, especially in the central and upper parts of the distribution.
$D^{(1,1)}$	Stretching-transition share	Are local timing-gap changes directionally balanced?	Stretching and compression occur with comparable frequency in both regimes.
$D^{(1,1)}$	Persistent-run share	Are local timing-gap changes isolated or temporally clustered?	Compression transitions are more often grouped into persistent episodes than stretching transitions.
$D^{(1,1)}$	Persistent episode length	How long do persistent stretching and compression episodes last?	Persistent episodes remain short, with compression episodes slightly longer than stretching episodes.
$D^{(1,1)}$	Persistent vs. non-persistent magnitude	Are persistent episodes also the strongest local distortions?	The strongest distortions are mostly non-persistent; persistent episodes are weaker but more regular within streams.

Table 5: Summary of the empirical readings provided by the Order-0 and Order-1 derived metrics.

This ratio compares the typical KuCoin timing gap with the typical spacing between KuCoin updates. As a second summary, we also use the persistent compression share (14). It captures whether KuCoin compression is mostly isolated or organized into same-sign episodes. This relates to the denominator of Equation (15), since persistent compression shortens the KuCoin intervals used by the latest-update matching rule.

For each of the 550 symbol–regime pairs with at least 1000 attempted matches, we compute the mismatch rate and report it as a function of these two stream-level summaries in Figure 1. In both panels, mismatch rates increase with the corresponding summary. The rise is limited at first, but becomes sharp once ρ_K exceeds roughly 2, or once the persistent-compression share exceeds roughly 0.5. The practical point is that, at the stream level, summaries derived from the temporal transformation already distinguish cases where mismatch rates remain contained from cases where the mismatch-rate distribution develops a large upper tail.

We now move to the level of individual Binance updates and ask whether the pipeline transformation can change the interpretation of the downstream signal. For the dislocation signal, a simple operational reading comes from its sign: it indicates which exchange appears priced above the other at the matched timestamp. We therefore focus on cases where this sign is reversed by the pipeline transformation, and define the sign-flip indicator

$$\mathrm{flip}_i = \mathbf{1}\{\mathrm{sign}(d_j^i) \neq \mathrm{sign}(d_j^o)\}\,. \tag{16}$$

We then estimate the empirical probability of a sign flip across the 2.8 million individual Binance updates in the 550 retained symbol–regime pairs. We look at how this probability changes with two local quantities.

The first is the relative Order-0 transformation between the Binance event B_i and the KuCoin event K_j, measured by $|D_i^B - D_j^K|$. It captures how much the two timing gaps diverge at the matched update. We compute deciles within each stream, pool updates by decile rank across streams, and report the fraction of updates with $\mathrm{flip}_i = 1$ in each pooled decile. The left panel of Figure 2 links this local relative transformation to the downstream signal. lthough sign flips remain rare, they become much more likely when the Binance and KuCoin timing gaps diverge in magnitude. The probability rises from below 0.1% in the lowest decile of $|D_i^B - D_j^K|$ to about 1.3% in the highest.

The second quantity is the signed length of the episode containing K_j in the KuCoin stream. In the right panel of Figure 2, positive lengths correspond to stretching episodes and negative lengths to compression episodes. For short episodes, roughly between -4 and 4 transitions, the sign-flip probability remains low and nearly flat. It increases outside this central region, especially for longer compression episodes, reaching about 2.6% at the left end of the reported range. The key point is that the Order-1 run structure separates a stable local region from a more fragile one. In the fragile region, persistent compression narrows the KuCoin matching intervals and makes the downstream sign more exposed to temporal reconstruction.

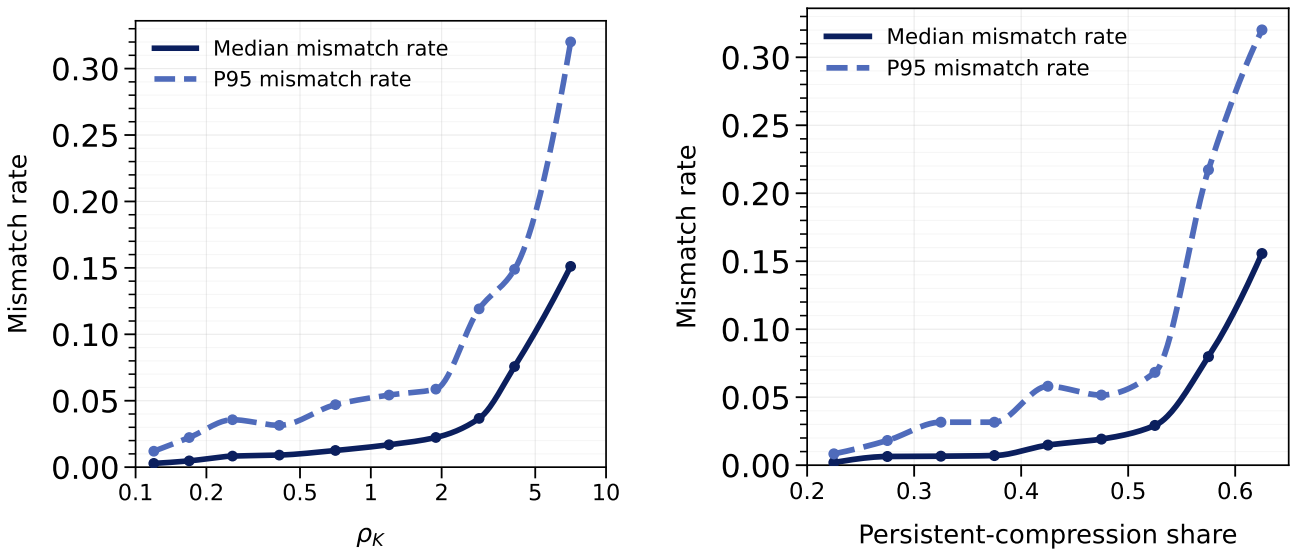

Figure 1: Stream-level mismatch rate as a function of two aggregate summaries of the temporal transformation.

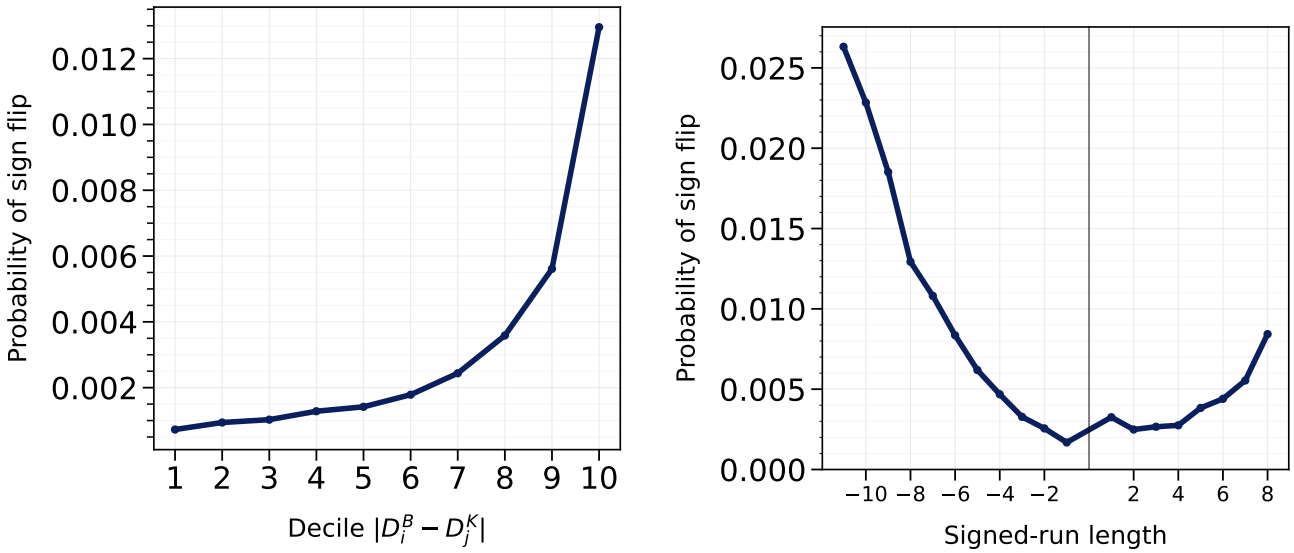

Figure 2: Local sign-flip probability as a function of two local summaries of the temporal transformation.

3.6.3 Practical implications

The analysis above shows that temporal structures measured inside the pipeline can identify contained and fragile regions both at the stream level and at the level of individual matched events. These metrics describe the mechanism itself before any downstream signal is chosen, and the dislocation example shows how this mechanism can propagate into a downstream signal. In a real-time system, this means that two pipelines processing the same market updates may expose downstream logic to different signals, and therefore potentially to different reactions.

This raises a practical question: how can such effects be limited? There are two complementary levels of response. The first acts on the pipeline itself, by reducing the temporal transformations that create fragile alignment conditions. This is the structural response. The concrete correction is pipeline-specific, but the metrics identify which temporal effect must be reduced.

The second acts at the downstream signal level. It is better understood as a guardrail than as a structural correction. An online rule can decide not to use a matched update when the selected event is too old or unsuitable for the decision being made. For example, if the KuCoin update selected online is denoted by K_l, a simple freshness rule to reduce mismatches would require

$$0 \le r_i^B - r_l^K \le TTL_{\text{input}}.$$

Such a rule is useful, but it rejects or down-weights observations. It does not remove the temporal transformation that produced the fragile alignment and trades signal coverage against the risk of using stale data. This is also why, in a real-time system, the operational question is not necessarily whether the number of mismatches is minimized. If a more recent KuCoin update is available when the Binance update is processed, the latest-available rule may be the intended behavior. The relevant question is often whether the selected update is fresh enough and appropriate for the decision.

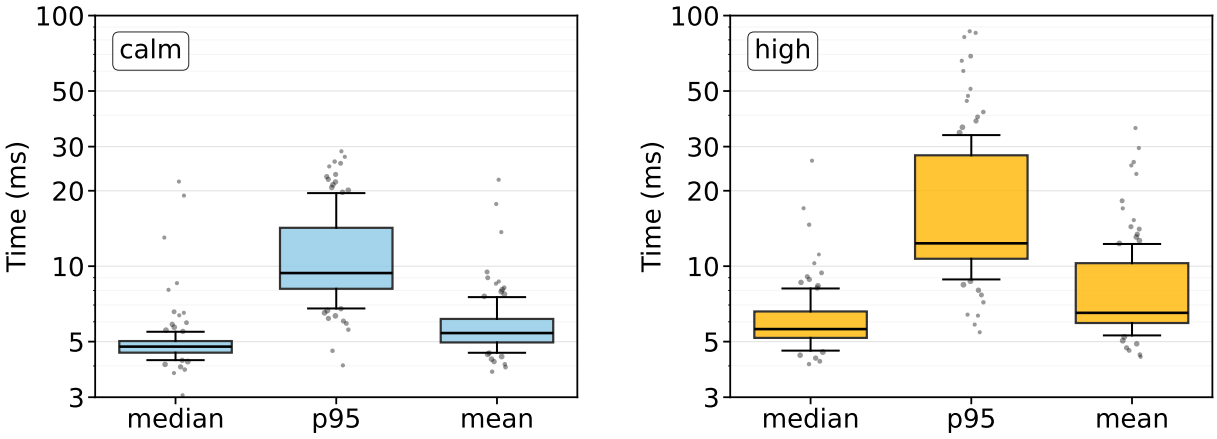

Figure 3: Cross-stream distributions of classical order-0 delay summaries by regime: median, P95, and mean $D^{(0)}$

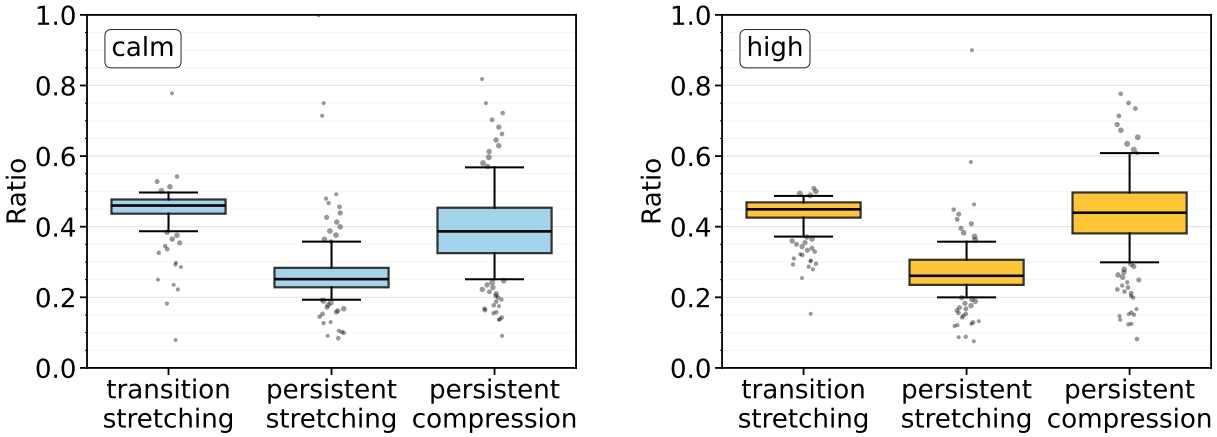

Figure 4: Cross-stream distributions of first-order run metrics by regime: stretching transition share, persistent stretching share, and persistent compression share.

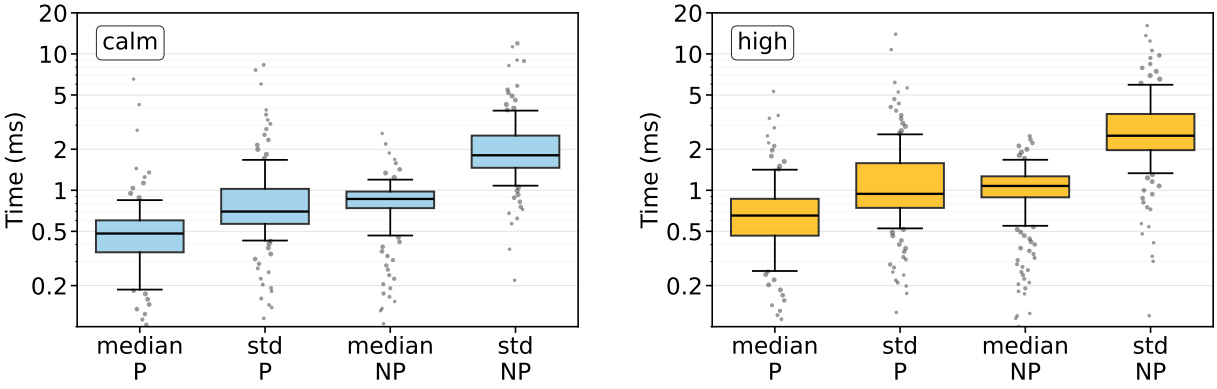

Figure 5: Cross-stream distributions of first-order magnitude summaries by regime: median and standard deviation of $D^{(1,1)}$ for persistent (P) and non-persistent (NP) runs.

4 Conclusion

This case study belongs to a broader class of real-time systems in which the value of information depends on when it becomes available. In such systems, a pipeline is best understood as a temporal transformation from an input stream to an output stream.

The case study shows why this matters. When the scale of the transformation is comparable to the scale of the phenomenon being measured, the pipeline starts to shape the set of downstream signals that can be observed. In the multi-exchange example, the same input updates can lead to different cross-exchange alignments, different dislocation values, and different signal signs after the pipeline transformation. This creates practical constraints for real-time systems. Replays, tests, and calibration procedures must preserve the relevant transformation if they are meant to reproduce the signals available to downstream decision logic.

The framework developed in this note addresses this problem by making the temporal transformation itself measurable. It exposes how timing gaps evolve, how local stretching and compression are organized, and where these effects become large enough to affect downstream signals. More importantly, it provides a way of thinking about the system: separating the temporal transformation induced by the pipeline, the observation support on which it is measured, and the assumptions needed to interpret the resulting metrics. Once characterized, these timing effects become explicit design variables for the system and for the downstream signals built on top of it.

A Appendix

A.1 Timestamp uncertainty

This appendix describes how timestamping error is measured for each node in Section 3. Over the observation window, we estimate two empirical quantities across all nodes: the observed rate of change of the timestamp error between successive synchronization states, and a practical absolute timestamp error relative to the reference source.

Methodology. Each node synchronizes its clock to a common NTP reference. In this implementation, synchronization diagnostics are collected from Chrony during the observation window. A lightweight collector runs on each node and records the local clock offset reported relative to the reference source. The collector follows the synchronization update cadence. Before each sample, it reads the current update interval Δ_{sync}. The delay before the next sample is then set to $\Delta_{\text{sample}} = \lceil 1.10\,\Delta_{\text{sync}} \rceil$. This margin makes the next read occur shortly after the next synchronization update, instead of repeatedly recording the same state.

For each node c, the collector produces a sequence of offset samples $(\tau_m, \hat{e}_c(\tau_m), \epsilon_c(\tau_m))_m$, where τ_m is the sampling time, $\hat{e}_c(\tau_m)$ is the observed clock offset relative to the reference source, and $\epsilon_c(\tau_m)$ is the reported uncertainty of that offset measurement. To estimate how quickly the clock error changes, we assume that the error evolves smoothly between two successive sampled synchronization states. Under this assumption, we define the empirical clock-error variation coefficient for node c as

$$\hat{\lambda}_c = \max_m \frac{|\hat{e}_c(\tau_{m+1}) - \hat{e}_c(\tau_m)| + \epsilon_c(\tau_{m+1}) + \epsilon_c(\tau_m)}{\tau_{m+1} - \tau_m}.$$

The numerator accounts both for the observed offset change and for the reported uncertainty of the two offset measurements involved in the finite difference. The global coefficient used in the case study is then obtained by taking the maximum across nodes:

$$\hat{\lambda} = \max_c \hat{\lambda}_c. \tag{17}$$

We also estimate the elementary timestamp-error bound η_c relative to the latent reference. For each offset sample, we define

$$B_c(\tau_m) = |\hat{e}_c(\tau_m)| + \epsilon_c(\tau_m), \tag{18}$$

where $\hat{e}_c(\tau_m)$ is the observed offset and $\epsilon_c(\tau_m)$ is the reported uncertainty of that offset measurement. For each node c, we estimate

$$\hat{\eta}_c = \max_m B_c(\tau_m). \tag{19}$$

The global timestamp-error bound used in the case study is then

$$\hat{\eta} = \max_c \hat{\eta}_c. \tag{20}$$

This bound is used as the empirical value of the elementary timestamp-error scale introduced in Equation (2).

Measures. We run the collector over the observation window. Figures 6 and 7 show the sampled quantities from which the global estimates $\hat{\lambda}$ and $\hat{\eta}$ are derived.

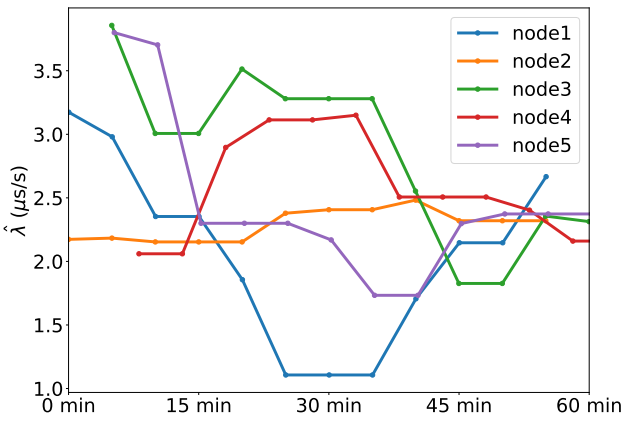

Figure 6: Empirical timestamp-error variation coefficient by node over the observation window.

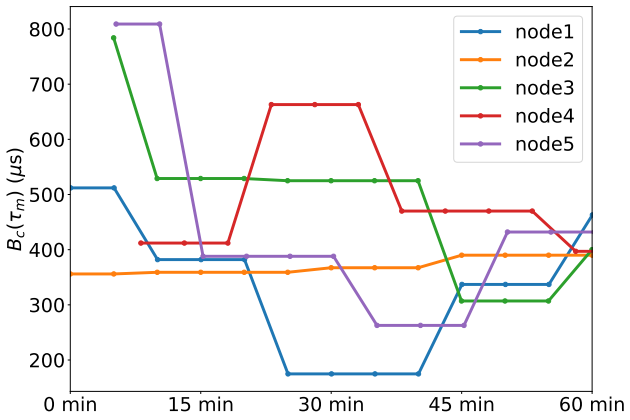

Figure 7: Reference-time timestamp-error quantity $B_c(\tau_m)$ by node over the observation window.

The largest node-level maximum timestamp-error bound is reached on node 3, with $\hat{\eta} = 784\,\mu$s. For timestamp-error variation, the largest empirical clock-error variation coefficient is also reached on node 3, with $\hat{\lambda} = 3.86\,\mu$s/s. These two empirical quantities are used in the case study: $2\hat{\eta} = 1568\,\mu$s for interpreting order-0 timing gaps, and $\hat{\lambda}$ for refined finite-difference uncertainty bounds.

The estimate of $\hat{\lambda}$ depends on the synchronization update cadence, since it is computed from offset changes between sampled synchronization states. It should therefore be read as an empirical scale for clock-error variation under the clock discipline used during the experiment. In the case study, the clocks share the same reference source, and the observed variation remains small compared with the temporal effects we analyze.

This measurement focuses on synchronization error relative to the reference source. Other timestamping effects, such as timestamp call overhead, scheduler effects, logging delay, or local buffering, are not modeled separately. They are treated as part of the fixed measurement setup.

References

[1] Charles Colella. Design trade-offs and failure modes in a real-time market data pipeline. Technical note No 1, May, 2026.

[2] Charles Colella. Real-time market data system. `https://github.com/charlescol/real-time-market-data-system`, 2026. GitHub repository.

[3] Carlo Demichelis and Philip Chimento. IP Packet Delay Variation Metric for IP Performance Metrics (IPPM). RFC 3393, RFC Editor, November 2002.